

Zero to Vibe Coding

코딩을 한 번도 해본 적 없는 사람을 위한
Python X ChatGPT X Codex 첫 개발 입문

기획·학습 기록: 이소연
공동 집필·기술 구성: ChatGPT

초판 베타 1.0 · 2026년 7월

목차

내용	쪽
이 책을 시작하기 전에	4
가장 먼저 할 일: ChatGPT 를 개인 튜터로 설정하기	5
PART 1. 컴퓨터에게 첫 명령 내리기	8
1 장. Python, ChatGPT, Codex 는 각각 무슨 일을 할까	9
2 장. Python 과 ChatGPT 데스크톱 앱 준비하기	10
3 장. 터미널은 무서운 검은 화면이 아니다	12
4 장. 첫 프로젝트 폴더 만들기	13
5 장. 첫 Python 파일 만들고 실행하기	14
6 장. 프로그램과 첫 대화하기: input 과 변수	16
PART 2. ChatGPT 와 Codex 를 내 개발 팀으로 만들기	18
7 장. ChatGPT 에게 결과를 보여주는 법	19
8 장. Codex 에서 프로젝트 열고 첫 수정을 맡기기	20
9 장. 프롬프트를 길게 잘 써야 한다는 부담 버리기	21
PART 3. 첫 프로젝트: 숫자 맞추기 게임	23
10 장. 게임을 Codex 에게 처음부터 맡겨보기	24
11 장. 숫자 게임 코드에서 다섯 가지만 읽기	24
12 장. 사용해보고 불편함을 기능으로 바꾸기	25
13 장. q 로 중간 종료하고 다시 플레이하기	27
14 장. 숫자 게임으로 실제로 배운 것	28

내용	쪽
PART 4. 두 번째 프로젝트: Todo 앱	30
15 장. 새 프로젝트는 새 폴더에서 시작하기	31
16 장. 리스트를 메모장처럼 이해하기	31
17 장. 삭제 기능은 짧은 요청으로 시작한다	33
18 장. 다음 문제: 프로그램을 켜다 켜면 목록이 사라진다	34
PART 5. GitHub 에 올려 포트폴리오로 만들기	35
19 장. GitHub 는 코드 전시장이자 변경 기록이다	36
20 장. 포트폴리오 README 는 결과보다 과정을 보여준다	37
21 장. 혼자 계속하기 위한 4 주 로드맵	38
맺음말. “대박!”이라고 말했던 순간을 기억하기	39
부록 A. ChatGPT 개인 튜터 프롬프트	40
부록 B. 상황별 짧은 Codex 프롬프트	40
부록 C. 터미널 명령어 cheatsheet	41
부록 D. 자주 만나는 오류와 첫 대응	42
부록 E. 초보자 용어 사전	42
부록 F. 공식 자료와 확인일	43

이 책을 시작하기 전에

“나는 코딩을 아무것도 모른다”에서 출발한다

이 책은 이미 코딩을 아는 사람이 초보자를 상상하며 만든 문법 요약집이 아니다. 실제로 Python 을 처음 실행하고, 터미널의 \$ 표시가 무엇인지 묻고, my-first-python 폴더 안에서 나가야 하는지 고민했던 한 초보자의 학습 기록에서 출발했다.

첫날의 질문은 아주 구체적이었다.

“아직 my-first-python 안에 들어와 있는 상태야. 여기서 안 나가도 돼?”

답은 “안 나가도 된다”였다. 오히려 그 폴더 안에 있어야 했다. 개발자에게는 너무 당연해 보여 교재에서 자주 생략되는 설명이지만, 처음 시작하는 사람에게는 바로 이런 지점이 가장 중요하다.

이 책은 그런 질문을 생각하지 않는다. 터미널을 여는 방법, 명령을 입력하고 Enter 를 누르는 것, 성공했는데 아무 메시지도 나오지 않는 경우, 파일이 실제로 어디에 만들어지는지부터 다룬다.

그리고 문법을 외우는 대신 다음 순서를 반복한다.

1. 아주 작은 목표를 정한다.
2. 직접 실행한다.
3. 결과를 확인한다.
4. 불편한 점을 찾는다.
5. ChatGPT 와 대화한다.
6. Codex 에게 파일 수정을 맡긴다.
7. 다시 실행해 확인한다.

이 반복이 이 책에서 말하는 **바이브코딩**이다.

이 책의 약속

- 한 번에 한 단계씩만 진행한다.
- 명령어를 보여줄 때는 어디에 입력하는지도 함께 말한다.
- 정상이라면 무엇이 보여야 하는지 알려준다.
- 오류가 나도 “망했다”고 생각하지 않도록 다음 행동을 제시한다.
- 처음부터 완벽한 프롬프트를 요구하지 않는다.
- AI 가 만든 코드를 무조건 믿지 않고 직접 실행하고 검토한다.
- 문법은 프로젝트에서 필요해진 순간에만 설명한다.

이 책의 범위

실습의 주 흐름은 macOS 를 기준으로 한다. 실제 출발점이 MacBook 과 Anaconda 환경이었기 때문이다. Windows 사용자를 위한 명령과 차이점도 각 장에 함께 적었다. 화면의 버튼 이름은 앱 업데이트에 따라 조금 달라질 수 있다.

2026 년 7 월 기준 새 ChatGPT 데스크톱 앱에는 ChatGPT 의 Chat 과 Work, 그리고 Codex 가 함께 들어 있다. 기존 Codex 앱 사용자는 업데이트 후 새 앱에서 Codex 를 계속 사용할 수 있다. 처음 설치하는 사용자는 공식 ChatGPT 다운로드 페이지에서 운영체제에 맞는 앱을 설치하면 된다.

Python 버전 숫자는 시간이 지나면 달라진다. 이 책은 특정 숫자를 외우게 하지 않고 공식 다운로드 페이지에 표시되는 지원 버전을 사용하도록 안내한다.

준비물

- 인터넷에 연결된 macOS 또는 Windows 컴퓨터
- ChatGPT 계정
- Python
- ChatGPT 데스크톱 앱의 Codex
- 실수해도 다시 해보겠다는 마음

유료 요금제가 반드시 필요한지는 계정과 기능 제공 범위에 따라 달라질 수 있다. 앱에서 Codex 가 보이지 않으면 먼저 앱을 최신 버전으로 업데이트하고 계정의 이용 가능 여부를 확인한다.

가장 먼저 할 일: ChatGPT 를 개인 튜터로 설정하기

이 책은 혼자 읽는 책이라기보다, ChatGPT 와 함께 진행하는 실습 안내서에 가깝다. 새 ChatGPT 대화를 열고 아래 프롬프트를 그대로 붙여넣는다.

복사해서 붙여넣을 튜터 프롬프트

나는 코딩을 한 번도 해본 적 없는 완전 초보자야.
앞으로 너는 내 Python & Codex 튜터 역할을 해줘.

내 목표는 Python 문법을 빨리 외우는 것이 아니라,
Codex 를 활용해 작은 프로그램을 직접 만들고 개선할 수 있는 사람이 되는 거야.

다음 규칙을 지켜줘.

1. 절대로 한 번에 많은 단계를 주지 마.
2. 항상 한 단계만 안내하고 내가 실행 결과를 보낼 때까지 기다려.
3. 명령어를 알려줄 때는 터미널, Python 화면, Codex 중 어디에 입력하는지 명확히 말해줘.
4. 내가 보게 될 예상 출력도 함께 보여줘.
5. 어려운 용어는 쉬운 비유와 함께 설명해줘.
6. 내가 질문한 내용이 아주 기초적이어도 생각하거나 무시하지 마.
7. 오류가 나면 오류 메시지를 그대로 보내도록 안내하고, 원인을 한 번에 하나씩 확인해줘.
8. Codex 용 프롬프트는 처음에는 내가 짧게 작성해주고, 점점 내가 직접 쓰도록 도와줘.
9. 내가 올린 .py 파일과 터미널 결과를 실제로 읽고 코드 리뷰를 해줘.
10. AI 가 코드를 수정한 뒤에는 내가 직접 실행해서 확인하도록 안내해줘.
11. 기존 파일 삭제, 개인정보 노출, 비밀키 업로드 같은 위험이 있으면 먼저 경고해줘.
12. 내가 완료했다고 말하면 그때 다음 단계로 넘어가.

항상 아래 순서로 진행해줘.

- 이번 한 단계의 목표
- 왜 하는지
- 내가 입력할 내용
- 정상일 때 예상 결과
- 문제가 생겼을 때 확인할 것

나는 macOS 사용자야. Anaconda 가 설치되어 있을 수도 있어.
먼저 Python 이 이미 실행되는지 확인하는 단계부터 시작해줘.

Windows 사용자라면 마지막 두 줄을 아래처럼 바꾼다.

나는 Windows 사용자야.
먼저 Python 이 이미 실행되는지 확인하는 단계부터 시작해줘.

파일도 함께 첨부한다

ChatGPT 가 현재 상황을 정확히 이해하려면 설명만 쓰는 것보다 실제 파일을 첨부하는 편이 좋다.

다음 중 현재 가지고 있는 것을 올린다.

- 이 저장소의 tutorial.py
- 지금 수정 중인 main.py
- 숫자 게임 파일인 number_game.py
- Todo 앱 파일인 main.py
- 오류가 난 터미널 화면의 스크린샷
- 프로젝트 폴더를 압축한 ZIP 파일

ChatGPT 입력창의 파일 첨부 버튼을 누르거나 파일을 대화창으로 끌어다 놓는다. 프로젝트 전체를 올릴 때는 **개인 문서 전체가 아니라 연습 프로젝트 폴더만** 압축한다.

절대 올리지 말 것

비밀번호, 주민등록번호, 회사 기밀, API 키, 인증 토큰, .env 파일, 은행 자료가 포함된 폴더는
첨부하지 않는다.

결과를 보내는 가장 좋은 방법

터미널에서 보인 내용을 처음부터 끝까지 복사해 보낸다.

```
(base) soyeonlee@MacBook:~/Desktop/my-first-python $ python main.py
이름을 입력하세요: 소연
안녕하세요! 소연
```

오류가 났다면 오류의 마지막 한 줄만 보내지 말고 가능한 범위에서 전체를 보낸다. 파일이 바뀌었다면
수정된 .py 파일도 함께 첨부한다.

PART 1. 컴퓨터에게 첫 명령 내리기

1 장. Python, ChatGPT, Codex 는 각각 무슨 일을 할까

처음에는 세 도구가 모두 “코딩을 해주는 것”처럼 보일 수 있다. 역할을 나누면 훨씬 덜 헛갈린다.

Python: 코드를 실제로 실행하는 사람

main.py 안에 다음 코드가 있다고 하자.

```
print("안녕하세요!")
```

Python 은 이 파일을 읽고 실제로 화면에 글자를 출력한다. 요리로 비유하면 레시피를 읽고 불을 켜서 음식을 만드는 조리사다.

ChatGPT: 설명하고 함께 생각하는 튜터

ChatGPT 는 다음 일을 맡는다.

- 지금 무엇을 해야 하는지 한 단계씩 설명한다.
- 오류 메시지를 읽고 원인을 추정한다.
- Codex 에게 보낼 요청을 다듬는다.
- 완성된 코드를 초보자 눈높이로 해설한다.
- 다음 기능을 기획하도록 질문한다.

ChatGPT 가 코드 예시를 보여줘도 그 코드가 내 컴퓨터의 파일에 자동으로 저장되는 것은 아니다. 파일을 직접 수정하거나 Codex 에 맡겨야 한다.

Codex: 내 프로젝트 폴더에서 일하는 AI 개발자

Codex 는 프로젝트 폴더를 열어 파일을 읽고, 수정하고, 터미널 명령을 실행해 테스트할 수 있다. 현재 ChatGPT 데스크톱 앱에서는 왼쪽 위 메뉴에서 Codex 보기를 선택해 사용할 수 있다. Codex 는 로컬 파일, 저장소, 터미널, 개발 도구를 사용해 소프트웨어 작업을 수행하도록 제공된다.

세 도구의 협업 구조

```

나: 무엇을 만들지 결정한다.
↓
ChatGPT: 다음 한 단계와 요청 방법을 설명한다.
↓
Codex: 실제 프로젝트 파일을 수정하고 테스트한다.
↓
Python: 수정된 코드를 실행한다.
  
```

↓
나: 결과를 사용해보고 다음 개선점을 찾는다.

이 책에서 가장 중요한 사람은 AI가 아니라 결과를 확인하고 다음 방향을 결정하는 나다.

2장. Python 과 ChatGPT 데스크톱 앱 준비하기

2-1. Python 이 이미 있는지 먼저 확인한다

설치부터 시작하기 전에 이미 Python 이 있는지 확인한다. Anaconda 를 설치한 적이 있다면 Python 이 함께 들어 있을 가능성이 높다.

macOS 에서 터미널 열기

1. 키보드에서 Command(⌘) + Space 를 누른다.
2. 검색창에 Terminal 또는 터미널을 입력한다.
3. Enter 를 누른다.

Windows 에서 PowerShell 열기

1. 시작 메뉴를 연다.
2. PowerShell 을 검색한다.
3. Windows PowerShell 을 실행한다.

정상적으로 보일 수 있는 화면

macOS에서는 (base) 이름@MacBook ~ % 또는 \$가 보일 수 있다. Windows에서는 PS C:\Users\이름> 같은 모양이 보인다. 모양이 완전히 같지 않아도 괜찮다.

터미널 또는 PowerShell 에 아래를 입력하고 Enter 를 누른다.

```
python --version
```

안 되면 아래를 시도한다.

```
python3 --version
```

다음처럼 버전이 나오면 설치되어 있다.

```
Python 3.13.5
```

Anaconda 사용자는 다음처럼 보일 수 있다.

```
Python 3.13.5 | packaged by Anaconda, Inc.
```

이것도 정상이다. 처음에는 굳이 환경을 바꾸지 말고 그대로 진행한다.

2-2. Python 이 없다면 공식 페이지에서 설치한다

공식 다운로드 주소:

<https://www.python.org/downloads/>

Python 공식 문서는 가능하면 최신 지원 버전을 사용하도록 권장하며, python.org 에서 macOS 용 설치 패키지를 제공한다.

macOS

1. 공식 페이지에서 macOS 용 Download Python 버튼을 누른다.
2. 내려받은 .pkg 파일을 연다.
3. 설치 안내를 따라 진행한다.
4. 설치가 끝나면 터미널을 완전히 닫았다가 다시 연다.
5. `python3 --version` 을 확인한다.

Windows

1. 공식 페이지에서 Windows 용 설치 안내를 선택한다.
2. 제공되는 설치 관리자 또는 설치 파일을 실행한다.
3. 화면에 PATH 관련 선택지가 보이면 Python 명령을 PATH 에 추가하는 옵션을 활성화한다.
4. PowerShell 을 다시 열고 `python --version` 을 확인한다.

버전 숫자가 책과 달라도 3 으로 시작하는 지원 버전이면 대부분의 실습을 진행할 수 있다.

2-3. ChatGPT 데스크톱 앱 설치하기

공식 다운로드 주소:

<https://chatgpt.com/download/>

2026 년 7 월 기준 새 데스크톱 앱은 macOS 와 Windows 에서 제공되며, ChatGPT 와 함께 Codex 를 포함한다. 기존 Codex 앱 사용자는 앱을 업데이트하면 기존 프로젝트를 유지하면서 새 ChatGPT 앱의 Codex 보기로 이동할 수 있다.

1. 운영체제에 맞는 앱을 내려받는다.
2. 앱을 설치하고 ChatGPT 계정으로 로그인한다.
3. 앱 왼쪽 위 메뉴에서 Codex 를 찾는다.

화면이 책과 다르다면

앱은 업데이트된다. Add project, Open folder, Codex, Projects 처럼 비슷한 의미의 메뉴를 찾는다. 정확한 버튼 위치보다 “프로젝트 폴더를 선택하고 Codex 대화를 연다”는 목적이 중요하다.

3장. 터미널은 무서운 검은 화면이 아니다

터미널은 컴퓨터와 글자로 대화하는 창이다. 오늘 사용할 명령은 몇 개뿐이다.

3-1. \$, %, > 뒤에 입력한다

다음처럼 보인다고 하자.

```
(base) soyeonlee@Soyeonui-MacBookAir.local:~ $
```

맨 마지막 \$ 뒤가 입력 위치다. (base)는 Anaconda 의 기본 환경이 켜져 있다는 뜻이다. 지금은 그대로 두어도 된다.

3-2. Python 대화형 화면 들어가기

터미널에 아래를 입력한다.

```
python
```

안 되면:

```
python3
```

다음처럼 >>>가 나오면 성공이다.

```
Python 3.13.5 | packaged by Anaconda, Inc. | ...
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

>>>는 Python 에게 한 줄씩 명령을 내리는 별도 화면이다.

3-3. 다시 일반 터미널로 나오기

>>> 뒤에 입력한다.

```
exit()
```

다시 \$, %, 또는 PS ...)가 보이면 일반 터미널로 돌아온 것이다.

초보자 질문: 왜 exit()에는 괄호가 있나요?

지금은 “Python 화면을 끝내는 명령의 정해진 모양”이라고 받아들이면 충분하다. 함수라는 개념은 나중에 실제로 필요해질 때 배운다.

3-4. 성공했는데 아무 말도 하지 않을 수 있다

터미널 명령은 성공하면 아무것도 출력하지 않는 경우가 많다. 예를 들어 폴더 이동이 성공해도 화면에 “성공!”이라고 나오지 않는다. 다음 명령으로 위치를 확인하면 된다.

3-5. 명령이 화면에서 두 줄로 보일 때

터미널 창이 좁으면 다음처럼 보일 수 있다.

```
$ python ma  
in.py
```

실제로 Enter 를 중간에 누르지 않았다면 화면에서 자동으로 줄바꿈된 것일 수 있다. 명령은 python main.py 한 줄이다.

4 장. 첫 프로젝트 폴더 만들기

프로젝트 폴더는 관련 파일을 모아두는 상자다. 개인 문서 전체나 바탕화면 전체를 Codex 프로젝트로 열지 않고, 연습 전용 작은 폴더를 만든다.

4-1. 바탕화면으로 이동하기

macOS:

```
cd ~/Desktop
```

Windows PowerShell:

```
cd $HOME\Desktop
```

cd 는 Change Directory, 즉 폴더 이동이라는 뜻이다.

아무 메시지도 나오지 않아도 괜찮다.

4-2. 새 폴더 만들기

macOS 와 Windows PowerShell 모두 다음을 사용할 수 있다.

```
mkdir my-first-python
```

mkdir 은 Make Directory, 즉 새 폴더 만들기다.

4-3. 폴더 안으로 들어가기

```
cd my-first-python
```

4-4. 현재 위치 확인하기

macOS:

```
pwd
```

예상 결과:

```
/Users/사용자이름/Desktop/my-first-python
```

Windows PowerShell에서는 `pwd` 를 입력하면 현재 경로가 표 형태로 표시될 수 있다.

초보자 질문: 폴더 안에 들어왔는데 다시 나가야 하나요?

나가지 않는다. 오히려 지금 이 폴더 안에서 계속 작업하는 것이 맞다. 여기서 `main.py` 를 만들면 파일도 이 폴더 안에 생기고, `python main.py` 도 이 위치에서 가장 쉽게 실행할 수 있다.

4-5. 폴더 안의 파일 보기

macOS 와 PowerShell:

```
ls
```

아직 아무 파일도 없다면 아무것도 나오지 않을 수 있다.

4-6. 오늘의 명령어 카드

명령	뜻	기억하는 방법
<code>cd</code> 폴더	폴더로 이동	change directory
<code>mkdir</code> 이름	새 폴더 만들기	make directory
<code>pwd</code>	현재 위치 보기	print working directory
<code>ls</code>	현재 폴더 내용 보기	list

5 장. 첫 Python 파일 만들고 실행하기

5-1. 빈 파일 만들기

macOS:

```
touch main.py
```

Windows PowerShell:

```
New-Item main.py
```

파일이 생겼는지 확인한다.

```
ls
```

예상 결과:

```
main.py
```

.py 는 Python 코드 파일이라는 뜻이다.

5-2. 파일 열기

macOS 에서 TextEdit 으로 연다.

```
open -e main.py
```

Windows 에서는 다음 중 하나를 사용한다.

```
notepad main.py
```

또는 파일 탐색기에서 main.py 를 메모장으로 연다.

터미널은 닫지 않는다

코드를 저장한 뒤 같은 터미널에서 실행할 것이므로 창을 그대로 둔다.

5-3. 첫 코드 입력하기

main.py 에 아래 한 줄을 입력한다.

```
print("안녕하세요! 첫 번째 파이썬 프로그램입니다.")
```

macOS 는 Command(⌘) + S, Windows 는 Ctrl + S 로 저장한다.

print 는 괄호 안의 내용을 화면에 출력한다.

5-4. 프로그램 실행하기

터미널로 돌아와 입력한다.

```
python main.py
```

안 되면:

```
python3 main.py
```

예상 결과:

```
안녕하세요! 첫 번째 파이썬 프로그램입니다.
```

이 문장이 보였다면 첫 프로그램을 만든 것이다.

5-5. 자주 만나는 오류

No such file or directory

현재 폴더에 main.py 가 없다는 뜻일 가능성이 크다.

```
pwd  
ls
```

두 명령으로 현재 위치와 파일 목록을 확인한다.

command not found: python

python3 main.py 를 시도한다.

코드가 바뀌지 않는다

파일을 저장했는지 확인한다. TextEdit 이나 메모장에 변경 표시가 남아 있으면 저장되지 않았을 수 있다.

따옴표 오류

다음처럼 따옴표가 한쪽만 있으면 오류가 난다.

```
| print("안녕하세요!)
```

열었으면 닫아야 한다.

6 장. 프로그램과 첫 대화하기: input 과 변수

이제 프로그램이 항상 같은 말만 하는 대신 사용자에게 이름을 묻게 한다.

파일을 다시 연다.

```
| open -e main.py
```

기존 내용을 아래 두 줄로 바꾼다.

```
| name = input("이름을 입력하세요: ")  
| print("안녕하세요!", name)
```

저장하고 실행한다.

```
| python main.py
```

예상 흐름:

```
| 이름을 입력하세요: 소연  
| 안녕하세요! 소연
```

변수는 이름표가 붙은 상자

```
| name = input("이름을 입력하세요: ")
```

사용자가 소연이라고 입력하면 Python 은 그 값을 name 이라는 이름으로 기억한다.

```
| name 상자 → "소연"
```

프로그래밍에서 =은 수학의 “양쪽이 같다”보다 “오른쪽 값을 왼쪽 이름에 저장한다”에 가깝다.

오늘 여기까지만 이해하면 된다

- `input()`은 사람의 입력을 받는다.
- `name`은 입력을 기억하는 이름이다.
- `print()`는 결과를 보여준다.

PART 2. ChatGPT 와 Codex 를 내 개발 팀으로 만들기

7 장. ChatGPT 에게 결과를 보여주는 법

ChatGPT 와 대화형 튜토리얼을 할 때는 “됐어”라고만 말해도 다음 단계로 갈 수 있지만, 문제가 생겼을 때는 실제 결과를 보여주는 습관이 중요하다.

터미널 결과 복사하기

터미널에서 명령과 출력 부분을 드래그해 복사한 뒤 ChatGPT 에 붙여넣는다.

```
(base) user@MacBook:~/Desktop/my-first-python $ python main.py
이름을 입력하세요: 소연
안녕하세요! 소연
```

파일 첨부하기

현재 파일이 main.py 라면 ChatGPT 대화창에 그 파일을 첨부하고 이렇게 말한다.

```
이 파일이 지금 내가 만든 코드야.
초보자 기준으로 실제 내용을 읽고,
이번 단계에서 이해해야 할 부분만 설명해줘.
```

프로젝트 폴더 전체 첨부하기

파일이 여러 개라면 프로젝트 폴더를 ZIP 으로 압축해 올릴 수 있다.

macOS Finder 에서:

1. 프로젝트 폴더를 마우스 오른쪽 버튼으로 누른다.
2. “압축”을 선택한다.
3. 만들어진 .zip 파일을 ChatGPT 에 첨부한다.

Windows 파일 탐색기에서는 폴더를 오른쪽 클릭하고 압축 ZIP 파일 만들기를 선택한다.

스크린샷보다 텍스트가 좋은 경우

오류 메시지는 가능하면 텍스트로 복사한다. ChatGPT 가 명령과 경로를 더 정확히 읽을 수 있다. 화면 배치나 버튼 위치가 문제라면 스크린샷을 함께 보낸다.

8장. Codex 에서 프로젝트 열고 첫 수정을 맡기기

8-1. 프로젝트 폴더 열기

ChatGPT 데스크톱 앱에서 Codex 보기를 선택한다. 프로젝트 추가 또는 폴더 열기 메뉴에서 다음 폴더를 고른다.

```
Desktop/my-first-python
```

Codex 에 개인 홈 폴더 전체, 문서 폴더 전체, 바탕화면 전체를 열어주지 않는다. 작은 프로젝트 폴더만 선택한다.

8-2. 첫 요청은 그대로 복사해도 된다

Codex 입력창에 붙여넣는다.

```
나는 파이썬을 배우는 초보자야.

현재 프로젝트의 main.py 를 수정해줘.
현재는 이름만 입력받는데 이름과 나이를 입력받도록 바꿔줘.

기존 파일은 삭제하지 마.
수정한 뒤 직접 실행해서 오류가 없는지 확인해줘.
마지막에는 변경한 코드를 초보자도 이해할 수 있게 설명해줘.
```

Codex 는 대략 다음 일을 한다.

1. 현재 main.py 를 읽는다.
2. 코드를 수정한다.
3. 터미널에서 실행해본다.
4. 변경 내용과 테스트 결과를 설명한다.

수정된 코드는 다음과 비슷하다.

```
name = input("이름을 입력하세요: ")
age = input("나이를 입력하세요: ")

print()
print(f"안녕하세요, {name}님!")
print(f"{age}살이시군요.")
print("좋은 하루 보내세요!")
```

8-3. Codex 가 “완료”했다고 해도 직접 실행한다

터미널에서:

```
python main.py
```

직접 이름과 나이를 입력한다. AI의 테스트와 사용자의 실제 사용은 다를 수 있다.

8-4. 변경 파일과 명령을 확인한다

Codex가 변경 승인이나 실행 승인을 요청할 수 있다. 다음을 본다.

- 수정 대상이 main.py가 맞는가?
- 예상하지 않은 파일을 삭제하지 않는가?
- 프로젝트 폴더 밖을 건드리는 명령은 없는가?
- 설치 명령이 있다면 왜 필요한지 설명했는가?

모르겠다면 승인하기 전에 이렇게 묻는다.

이 명령이 무엇을 하는지 초보자도 이해할 수 있게 설명해줘.
프로젝트 폴더 밖의 파일은 수정하지 마.

9장. 프롬프트를 길게 잘 써야 한다는 부담 버리기

처음에는 완벽한 요청을 써야 AI가 제대로 일한다고 생각하기 쉽다. 실제로는 짧게 시작하고 결과를 보며 고치는 방식이 더 자연스럽다.

최소 프롬프트 네 줄

main.py를 수정해줘.
이름과 나이를 입력받게 해줘.
기존 기능은 유지해줘.
수정 후 직접 테스트해줘.

이 정도면 출발할 수 있다.

결과를 보고 한 줄 더 요청한다

빈 나이를 입력하면 안내문을 보여주고 다시 입력받게 해줘.

또 결과를 보고:

출력이 너무 붙어 보여. 빈 줄을 넣어 보기 좋게 바꿔줘.

이게 대화형 개발이다.

프롬프트의 네 가지 선택 요소

꼭 전부 넣을 필요는 없지만 막힐 때 참고한다.

1. 대상: 어떤 파일을 바꿀까?

2. **목표:** 무엇을 추가할까?
3. **유지:** 기존에 지켜야 할 것은?
4. **확인:** 무엇을 테스트할까?

대상: number_game.py
목표: q 를 입력하면 종료
유지: 점수와 입력 검사는 그대로
확인: q 와 Q 둘 다 테스트

실제 초보자의 중요한 피드백

“너가 쓴 건 너무 내용이 많아. 개발 지식이 없어서 그렇게까지 세부적으로 고려하질 못하겠어.”

이 말이 맞다. 세부 조건을 모두 미리 생각하는 것은 초보자의 의무가 아니다. 먼저 원하는 기능만 말한다.

할 일 삭제 기능을 추가해줘.
하나 삭제, 여러 개 삭제, 전체 삭제가 가능하게 해줘.

Codex 가 구현한 뒤 사용해보고, 필요한 안전장치를 추가한다.

전체 삭제 전에 정말 삭제할지 한 번 더 물어봐줘.

좋은 바이브코딩은 “한 번에 완벽한 문서”가 아니라 짧은 관찰과 수정의 반복이다.

PART 3. 첫 프로젝트: 숫자 맞추기 게임

10 장. 게임을 Codex 에게 처음부터 맡겨보기

이번에는 새 파일을 만든다. Codex 에 아래 요청을 붙여넣는다.

```
현재 프로젝트에 number_game.py 파일을 새로 만들어줘.

컴퓨터가 1 부터 100 사이의 숫자를 하나 정하고,
사용자가 맞힐 때까지 숫자를 입력하는 게임을 만들어줘.

입력한 숫자가 정답보다 작으면 "더 큰 숫자입니다."라고 알려주고,
정답보다 크면 "더 작은 숫자입니다."라고 알려줘.
정답이면 축하 메시지를 출력하고 종료해줘.

코드는 파이썬 초보자가 읽기 쉽게 작성해줘.
만든 후 직접 실행해서 테스트해줘.
```

Codex 가 만든 기본 코드는 다음과 비슷하다.

```
import random

answer = random.randint(1, 100)

print("숫자 맞추기 게임을 시작합니다!")
print("1 부터 100 사이의 숫자를 맞춰보세요.")

while True:
    guess = input("숫자를 입력하세요: ")
    guess = int(guess)

    if guess < answer:
        print("더 큰 숫자입니다.")
    elif guess > answer:
        print("더 작은 숫자입니다.")
    else:
        print("정답입니다! 축하합니다!")
        break
```

터미널에서 직접 실행한다.

```
python number_game.py
```

Codex 가 seq 1 100 | python3 number_game.py 처럼 낯선 명령으로 자동 테스트할 수도 있다. 1 부터 100 까지를 프로그램에 차례로 넣어 정답을 반드시 만나게 하는 테스트다. 이 명령을 지금 외울 필요는 없다. “이 테스트가 무엇을 확인했는지 설명해줘”라고 물으면 된다.

11 장. 숫자 게임 코드에서 다섯 가지만 읽기

코드 전체를 한 번에 이해하지 않는다. 다섯 조각만 읽는다.

11-1. 랜덤 도구 가져오기

```
import random
```

Python 에 준비된 무작위 숫자 기능을 가져온다.

11-2. 정답 만들기

```
answer = random.randint(1, 100)
```

1 부터 100 사이의 정수 하나를 뽑아 answer 에 저장한다.

11-3. 계속 반복하기

```
while True:
```

아래 코드를 계속 반복한다. 정답을 맞춰 break 를 만날 때까지 입력을 다시 받는다.

11-4. 글자를 숫자로 바꾸기

```
guess = input("숫자를 입력하세요: ")  
guess = int(guess)
```

input()으로 받은 값은 처음에는 글자다. int()가 정수로 바꾼다.

11-5. 조건에 따라 다르게 행동하기

```
if guess < answer:  
    print("더 큰 숫자입니다.")  
elif guess > answer:  
    print("더 작은 숫자입니다.")  
else:  
    print("정답입니다! 축하합니다!")  
    break
```

- if: 첫 조건이 맞으면 실행
- elif: 첫 조건은 아니지만 이 조건이 맞으면 실행
- else: 앞의 조건이 모두 아니면 실행
- break: 반복 종료

정답보다 작지도 크지도 않다면 같은 숫자이므로 정답이다.

12 장. 사용해보고 불편함을 기능으로 바꾸기

게임을 직접 해보면 다음 불편함을 발견할 수 있다.

- abc 를 입력하면 프로그램이 꺼진다.
- 0 이나 101 을 넣어도 시도 횟수로 처리될 수 있다.
- 몇 번 만에 맞혔는지 알 수 없다.
- 점수가 없다.

이제 사용자는 기획자가 된다.

짧게 요청하기

```
number_game.py를 수정해줘.
글자나 1~100 밖의 숫자를 입력해도 종료되지 않게 해줘.
몇 번 만에 맞혔는지와 점수도 보여줘.
기존 게임은 유지하고 테스트해줘.
```

글자 입력을 막는 try 와 except

Codex 가 다음 구조를 추가할 수 있다.

```
try:
    guess = int(guess)
except ValueError:
    print("1 부터 100 사이의 정수를 입력해주세요.")
    continue
```

의미는 다음과 같다.

```
일단 정수로 바꿔본다.
실패하면 프로그램을 죽이지 않고 안내한다.
그리고 반복문의 처음으로 돌아간다.
```

continue 는 이번 반복의 나머지를 건너뛰고 다시 입력받게 한다.

범위 검사

```
if guess < 1 or guess > 100:
    print("1 부터 100 사이의 정수를 입력해주세요.")
    continue
```

숫자이긴 하지만 범위를 벗어난 입력을 걸러낸다.

시도 횟수와 점수

```
attempts = attempts + 1
score = 100 - (attempts - 1) * 5

if score < 0:
    score = 0
```

첫 번째 시도는 감점하지 않으므로 (attempts - 1)을 사용한다.

시도 횟수	계산	점수
1	$100 - 0 \times 5$	100
2	$100 - 1 \times 5$	95
3	$100 - 2 \times 5$	90
21 이상	0 보다 작으면 0 으로 고정	0

테스트할 입력

```
abc
3.5
0
101
50
```

앞의 네 입력에서 프로그램이 종료되지 않고 안내문이 나오는지 확인한다. 잘못된 입력은 시도 횟수에 포함하지 않는지도 본다.

13장. q 로 중간 종료하고 다시 플레이하기

게임을 하다가 그만두고 싶을 수 있다. 숫자 입력창에서 q 또는 Q 를 입력하면 종료하도록 요청한다.

```
number_game.py 에 중간 종료 기능을 추가해줘.
숫자 입력 중 q 나 Q 를 입력하면 이번 게임을 끝내고 정답을 보여줘.
q 는 시도 횟수에 포함하지 마.
기존 기능은 유지하고 테스트해줘.
```

핵심은 int() 보다 먼저 q 를 검사하는 것이다.

```
raw_guess = input("숫자를 입력하세요: ")

if raw_guess.lower() == "q":
    print(f"게임을 중단합니다. 정답은 {answer}였습니다.")
    break

try:
    guess = int(raw_guess)
except ValueError:
    ...
```

lower() 는 Q 도 소문자 q 로 바꿔 비교할 수 있게 한다.

다시 플레이 기능 요청

```
게임이 끝나면 "다시 플레이하시겠습니까? (y/n)"라고 물어봐줘.
y 나 Y 면 새로운 정답으로 다시 시작하고,
n 이나 N 이면 프로그램을 종료해줘.
새 게임에서는 시도 횟수와 점수를 초기화해줘.
기존 기능은 유지하고 테스트해줘.
```

여기서 중요한 조건은 새 게임의 상태 초기화다.

```
새 정답
attempts = 0
score = 100
```

완성된 게임 구조

완성 코드는 저장소의 다음 파일에 있다.

```
examples/02-number-game/03_final_replay.py
```

직접 실행한다.

```
python examples/02-number-game/03_final_replay.py
```

최종 테스트 체크리스트

- ☐ 글자를 입력해도 종료되지 않는다.
- ☐ 소수를 입력해도 종료되지 않는다.
- ☐ 0 과 101 을 거절한다.
- ☐ 정상 숫자만 시도 횟수로 계산한다.
- ☐ 점수가 0 아래로 내려가지 않는다.
- ☐ q 와 Q 가 모두 동작한다.
- ☐ 중간 종료 시 정답을 보여준다.
- ☐ y 와 Y 로 새 게임을 시작한다.
- ☐ n 과 N 으로 종료한다.
- ☐ 새 게임의 시도 횟수와 점수가 초기화된다.

14 장. 숫자 게임으로 실제로 배운 것

표면적으로는 if, while, random 을 배웠다. 더 중요한 학습은 다음과 같다.

사용해봐야 요구사항이 생긴다

처음부터 글자 입력, 점수, 종료, 재시작을 모두 떠올릴 필요는 없었다. 기본 게임을 사용해본 뒤 불편함을 하나씩 발견했다.

오류는 실패가 아니라 다음 요구사항이다

abc 를 입력했더니 ValueError 가 났다면 “나는 코딩에 소질이 없다”가 아니라 “잘못된 입력을 처리하는 기능이 필요하다”라고 해석한다.

테스트는 AI 에게만 맡기지 않는다

Codex 의 자동 테스트는 유용하지만, 사람이 실제로 입력하면서 느끼는 불편함까지 모두 찾지는 못한다.

프롬프트는 성장한다

처음에는 제공된 프롬프트를 복사했다. 다음에는 “중간 종료가 있으면 좋겠어”라고 기능을 직접 제안했다. 그다음에는 테스트 조건까지 적었다. 이것이 자연스러운 발전 순서다.

PART 4. 두 번째 프로젝트: Todo 앱

15 장. 새 프로젝트는 새 폴더에서 시작하기

숫자 게임과 Todo 앱은 목적이 다르다. 새 폴더를 만든다.

macOS:

```
cd ~/Desktop
mkdir todo-app
cd todo-app
pwd
```

Windows PowerShell:

```
cd $HOME\Desktop
mkdir todo-app
cd todo-app
pwd
```

Codex 에서도 todo-app 폴더를 새 프로젝트로 연다.

왜 새 폴더인가요?

파일이 섞이지 않고, Codex 가 어떤 프로젝트를 수정해야 하는지 명확해진다. 나중에 GitHub 에도 프로젝트별 저장소로 올리기 쉽다.

첫 Todo 프롬프트

현재 프로젝트에 main.py 를 만들어줘.

터미널에서 실행되는 아주 간단한 Todo 앱을 만들고 싶어.
기능은 세 개만 넣어줘.

1. 할 일 추가
2. 할 일 목록 보기
3. 종료

할 일은 리스트에 저장해줘.
아직 파일 저장과 함수는 사용하지 마.
초보자가 읽기 쉽게 작성하고 직접 테스트해줘.

처음부터 삭제, 저장, 완료 처리, 날짜를 모두 넣지 않는다. 작게 작동하는 버전부터 만든다.

16 장. 리스트를 메모장처럼 이해하기

기본 Todo 앱은 다음처럼 시작한다.

```
todos = []
```

빈 리스트다. 여러 개의 할 일을 순서대로 담은 메모장이라고 생각한다.

```
todos.append("운동하기")
todos.append("책 읽기")
todos.append("장보기")
```

결과:

```
["운동하기", "책 읽기", "장보기"]
```

append 는 맨 뒤에 새 항목을 붙인다.

기본 코드

```
todos = []

while True:
    print()
    print("===== Todo 앱 =====")
    print("1. 할 일 추가")
    print("2. 할 일 목록 보기")
    print("3. 종료")

    choice = input("메뉴를 선택하세요: ")

    if choice == "1":
        todo = input("추가할 할 일을 입력하세요: ")
        todos.append(todo)
        print("할 일이 추가되었습니다.")

    elif choice == "2":
        print()
        print("===== 할 일 목록 =====")

        if len(todos) == 0:
            print("아직 할 일이 없습니다.")
        else:
            for i in range(len(todos)):
                print(f"{i + 1}. {todos[i]}")

    elif choice == "3":
        print("Todo 앱을 종료합니다.")
        break

    else:
        print("1, 2, 3 중에서 선택해주세요.")
```

오늘 이해할 것은 두 줄뿐이다

```
todos = []
```

여러 값을 넣을 빈 목록을 만든다.

```
todos.append(todo)
```

사용자가 입력한 할 일을 목록 뒤에 추가한다.

for, range, len 이 아직 낯설어도 괜찮다. 지금은 “목록 안의 항목을 하나씩 번호와 함께 보여주는 부분”이라고 읽으면 된다.

17 장. 삭제 기능은 짧은 요청으로 시작한다

처음 프롬프트는 이 정도면 충분하다.

```
현재 프로젝트의 main.py 를 수정해줘.

할 일 삭제 기능을 추가해줘.
1. 하나 삭제
2. 여러 개 삭제
3. 전체 삭제

기존 기능은 유지해줘.
잘못 입력해도 프로그램이 종료되지 않게 해줘.
```

이 요청은 모든 세부 사항을 미리 정하지 않는다. Codex 의 결과를 실행한 뒤 다음을 확인한다.

- 하나 삭제할 때 번호를 어떻게 입력하는가?
- 여러 개 삭제는 1,3,5 처럼 입력하는가?
- 없는 번호를 넣으면 어떻게 되는가?
- 전체 삭제 전에 확인하는가?
- 빈 목록에서 삭제를 선택하면 어떻게 되는가?

부족한 부분만 다시 요청한다.

```
전체 삭제 전에 "정말 삭제하시겠습니까? (y/n)"라고 한 번 더 물어봐줘.
여러 개 삭제할 때 사용자가 1,3,5 처럼 심표로 입력할 수 있게 해줘.
중복 번호나 없는 번호를 입력해도 프로그램이 종료되지 않게 해줘.
```

삭제에 사용되는 리스트 기능

하나 삭제:

```
deleted = todos.pop(index)
```

pop 은 해당 위치의 항목을 꺼내면서 목록에서 제거한다.

전체 삭제:

```
todos.clear()
```

clear 는 리스트 안의 항목을 모두 비운다.

여러 개 삭제는 번호가 바뀌지 않도록 큰 번호부터 지우는 것이 안전하다. 이 세부 알고리즘을 초보자가 처음부터 생각해내지 못해도 괜찮다. Codex 에게 “번호가 밀려 잘못 삭제되지 않도록 처리해줘”라고 요구하면 된다.

완성 예제는 다음 파일에 있다.

```
examples/03-todo-app/02_with_delete.py
```

18 장. 다음 문제: 프로그램을 켜다 켜면 목록이 사라진다

현재 리스트는 프로그램이 실행되는 동안만 존재한다. 종료하면 메모리가 비워지므로 다음 실행에는 할 일이 없다.

이제 자연스럽게 다음 기능이 필요해진다.

Todo 앱을 종료해도 할 일이 사라지지 않게 파일에 저장해줘.
 프로그램을 시작할 때 이전 할 일을 자동으로 불러와줘.
 초보자가 이해할 수 있는 가장 단순한 방식으로 만들어줘.

Codex 는 텍스트 파일이나 JSON 파일을 제안할 수 있다. 이 시점부터 코드가 길어지기 때문에 함수가 도움이 된다.

함수는 반복 작업에 이름을 붙인 묶음

예를 들어 다음과 같은 이름을 만들 수 있다.

```
def show_todos():
    ...

def save_todos():
    ...
```

함수는 시험을 위해 미리 외우는 문법이 아니다. 코드가 길어지고 같은 작업을 여러 번 해야 해서 필요해지는 정리 도구다.

이 책의 첫 목표는 여기까지다. 저장 기능은 다음 프로젝트 단계로 남긴다. 중요한 것은 “문법 순서”가 아니라 “앱을 사용하며 다음 필요를 발견하는 흐름”이다.

PART 5. GitHub 에 올려 포트폴리오로 만들기

19 장. GitHub 는 코드 전시장이자 변경 기록이다

GitHub 에 프로젝트를 올리면 변경 기록을 관리하고, 클라우드에 백업하고, 공개 저장소를 프로필에 보여줄 수 있다.

처음에는 Git 명령을 모두 배우지 않고 웹 브라우저로 업로드해도 된다.

19-1. GitHub 저장소 만들기

1. <https://github.com/>에 로그인한다.
2. 오른쪽 위 +를 누른다.
3. New repository 를 선택한다.
4. Repository name 에 zero-to-vibe-coding 또는 프로젝트 이름을 적는다.
5. 다른 사람에게 보여주려면 Public 을 선택한다.
6. 저장소를 만든다.

GitHub 공식 안내도 프로젝트별로 새 저장소를 만드는 방식을 권장한다.

19-2. 파일 업로드하기

1. 저장소 화면에서 Add file 을 누른다.
2. Upload files 를 선택한다.
3. 프로젝트 폴더의 파일과 하위 폴더를 끌어다 놓는다.
4. 아래의 commit 메시지에 변경 내용을 짧게 적는다.
5. Commit changes 를 누른다.

GitHub 공식 웹 업로드 절차도 같은 흐름을 안내한다.

주의: ZIP 만 올리지 않는다

사람들이 GitHub 화면에서 README 와 코드를 바로 볼 수 있도록 압축을 푼 파일과 폴더를 업로드한다. 배포용 ZIP 은 Release 에 별도로 올릴 수 있다.

19-3. GitHub Desktop 을 쓰는 방법

Git 명령이 부담스럽다면 GitHub Desktop 을 사용할 수 있다. GitHub Desktop 은 명령줄 없이 저장소 만들기, 커밋, 푸시를 할 수 있는 그래픽 앱이다.

기본 흐름:

1. GitHub Desktop 설치 및 로그인
2. Add an Existing Repository from your Hard Drive 또는 새 저장소 만들기
3. 변경 파일 확인
4. Summary 에 짧은 설명 작성
5. Commit
6. Publish repository 또는 Push origin

19-4. 절대 올리지 말아야 할 것

- 비밀번호와 인증 토큰
- API 키
- .env 파일
- 고객 정보와 회사 기밀
- 개인 사진이나 문서가 섞인 폴더

.gitignore 에 다음처럼 적어 민감하거나 불필요한 파일을 제외할 수 있다.

```
__pycache__/  
*.pyc  
.env  
.DS_Store
```

20 장. 포트폴리오 README 는 결과보다 과정을 보여준다

초보자의 프로젝트는 기능이 화려해서 가치 있는 것이 아니다. 어떤 문제를 발견했고, AI 와 어떻게 대화했고, 무엇을 검증했는지 보여주면 된다.

README 기본 구조

```
# 프로젝트 이름  
  
## 프로젝트 소개  
왜 만들었는지 한두 문장으로 설명합니다.  
  
## 주요 기능  
- 기능 1  
- 기능 2  
- 기능 3  
  
## 실행 방법  
'''bash  
python main.py  
'''  
  
## 개발 과정
```

1. 가장 작은 버전을 만들었습니다.
2. 직접 사용하며 불편한 점을 찾았습니다.
3. Codex 에 기능 추가를 요청했습니다.
4. 잘못된 입력과 종료 흐름을 테스트했습니다.

배운 점

- 프로젝트 폴더와 파일 경로
- Python 입력과 출력
- AI 에게 짧게 요청하고 반복 수정하는 방법

다음 개선 계획

- 파일 저장
- 화면 인터페이스
- 테스트 자동화

숫자 게임 포트폴리오 문장 예시

코딩 경험이 없는 상태에서 Python 실행과 터미널 명령부터 시작했습니다.
Codex 로 기본 숫자 맞추기 게임을 만든 뒤, 직접 플레이하며
잘못된 입력 처리, 점수, 중간 종료, 재시작 기능을 단계적으로 추가했습니다.
각 기능은 짧은 자연어 요청 → 코드 변경 검토 → 직접 실행 테스트의 순서로 개발했습니다.

Zero to Vibe Coding 프로젝트의 차별점

- 완전 초보자의 실제 질문에서 출발했다.
- Python 설치와 터미널 열기부터 설명한다.
- ChatGPT 용 개인 튜터 프롬프트를 제공한다.
- 사용자가 자신의 파일을 첨부해 대화형으로 학습한다.
- Codex 프롬프트를 복사하는 단계에서 직접 쓰는 단계로 이동한다.
- 문법보다 작은 성공, 사용 경험, 반복 개선을 강조한다.

21 장. 혼자 계속하기 위한 4 주 로드맵

1 주차: 익숙해지기

- 첫 파일 만들기
- print, input, 변수
- 숫자 맞추기 게임
- 오류 메시지를 ChatGPT 에 보내기

2 주차: Todo 앱

- 리스트
- 추가와 목록 보기
- 하나·여러 개·전체 삭제

- 파일 저장 요구사항 작성

3 주차: GitHub

- 프로젝트별 저장소 만들기
- README 작성
- 변경할 때마다 커밋하기
- 프로젝트 화면 캡처 추가

4 주차: 나만의 작은 프로젝트

아래 중 하나를 고른다.

- 독서 기록기
- 여행 준비 체크리스트
- 간단한 지출 기록기
- 운동 기록기
- 공부 타이머
- 파일 이름 정리 도구

처음 요청은 짧게 쓴다.

나는 [문제]를 해결하는 작은 Python 프로그램을 만들고 싶어.
터미널에서 실행되는 가장 단순한 버전부터 만들어줘.
기능은 [핵심 기능 2~3 개]만 넣어줘.
초보자가 이해하기 쉽게 작성하고 테스트해줘.

맷음말. “대박!”이라고 말했던 순간을 기억하기

첫 `print()`가 실행되었을 때, 입력한 이름이 다시 화면에 나왔을 때, Codex가 실제 파일을 수정했을 때 느낀 놀라움은 단순한 감탄이 아니다. 컴퓨터가 더 이상 닫힌 상자가 아니게 되는 순간이다.

이 책의 목표는 독자를 빠르게 전문 개발자로 만드는 것이 아니다. 다음 행동을 스스로 선택할 수 있게 만드는 것이다.

폴더를 만든다.
파일을 연다.
작은 기능을 요청한다.
직접 실행한다.
불편함을 찾는다.
다시 요청한다.
변경을 기록한다.

이 흐름을 반복할 수 있다면 이미 바이브코딩을 시작한 것이다.

부록 A. ChatGPT 개인 튜터 프롬프트

아래 프롬프트는 prompts/chatgpt-tutor-prompt.txt 에도 들어 있다.

나는 코딩을 한 번도 해본 적 없는 완전 초보자야.
앞으로 너는 내 Python & Codex 튜터 역할을 해줘.

내 목표는 Python 문법을 빨리 외우는 것이 아니라,
Codex 를 활용해 작은 프로그램을 직접 만들고 개선할 수 있는 사람이 되는 거야.

규칙:

1. 한 번에 한 단계만 안내한다.
2. 내가 실행 결과를 보내기 전에는 다음 단계로 넘어가지 않는다.
3. 명령을 어디에 입력하는지 명확히 말한다.
4. 예상 출력을 함께 보여준다.
5. 어려운 용어는 비유로 설명한다.
6. 기초 질문을 생략하지 않는다.
7. 오류 메시지와 파일을 실제로 읽고 답한다.
8. Codex 프롬프트는 짧게 시작한다.
9. AI 가 수정한 뒤에는 직접 실행하게 한다.
10. 개인정보, 비밀키, 파일 삭제 위험을 먼저 경고한다.

먼저 Python 이 이미 설치되어 있는지 확인하는 단계부터 시작해줘.

부록 B. 상황별 짧은 Codex 프롬프트

새 파일 만들기

현재 프로젝트에 [파일이름.py] 를 만들어줘.
[원하는 기능] 을 하는 가장 단순한 Python 프로그램으로 만들어줘.
초보자가 이해하기 쉽게 작성하고 직접 테스트해줘.

기존 기능 추가

[파일이름.py] 에 [기능] 을 추가해줘.
기존 기능은 유지하고 수정 후 테스트해줘.

오류 수정

아래 오류가 발생했어.
원인을 확인하고 최소한으로 수정해줘.
수정한 이유와 다시 실행하는 방법을 설명해줘.

[오류 메시지 붙여넣기]

코드 리뷰

이 파일을 초보자 기준으로 리뷰해줘.
지금 꼭 이해해야 할 부분 세 개만 골라 설명해줘.
오류 가능성과 위험한 파일 작업도 확인해줘.

너무 복잡해졌을 때

기능은 유지하면서 코드를 초보자가 읽기 쉽게 정리해줘.
한 번에 큰 구조 변경을 하지 말고,
변경 전후 차이를 설명하고 테스트해줘.

테스트 요청

이 프로그램을 실제 사용자처럼 테스트해줘.
정상 입력, 빈 입력, 글자 입력, 범위 밖 입력, 종료 흐름을 확인해줘.
발견한 문제는 바로 고치지 말고 먼저 목록으로 알려줘.

부록 C. 터미널 명령어 치트시트

목적	macOS / Linux	Windows PowerShell
현재 위치	<code>pwd</code>	<code>pwd</code>
파일 목록	<code>ls</code>	<code>ls</code>
폴더 이동	<code>cd 폴더이름</code>	<code>cd 폴더이름</code>
한 단계 위	<code>cd ..</code>	<code>cd ..</code>
홈 폴더	<code>cd ~</code>	<code>cd \$HOME</code>
바탕화면	<code>cd ~/Desktop</code>	<code>cd \$HOME\Desktop</code>
폴더 만들기	<code>mkdir 이름</code>	<code>mkdir 이름</code>
빈 파일	<code>touch main.py</code>	<code>New-Item main.py</code>
Python 버전	<code>python --version</code> 또는 <code>python3 --version</code>	<code>python --version</code>
프로그램 실행	<code>python main.py</code> 또는 <code>python3 main.py</code>	<code>python main.py</code>
화면 지우기	<code>clear</code>	<code>cls</code>

부록 D. 자주 만나는 오류와 첫 대응

command not found: python

```
python3 --version  
python3 main.py
```

can't open file ... main.py

```
pwd  
ls
```

현재 위치와 파일 존재 여부를 확인한다.

SyntaxError

괄호, 따옴표, 콜론, 들여쓰기를 확인한다. 오류가 가리키는 줄뿐 아니라 바로 윗줄도 본다.

IndentationError

Python은 들여쓰기로 코드 묶음을 구분한다. 탭과 공백이 섞였거나, if, while, for 아래 줄이 안쪽으로 들어가지 않았을 수 있다.

ValueError: invalid literal for int()

글자를 정수로 바꾸려 했다는 뜻이다. try와 except로 안내문을 보여주고 다시 입력받을 수 있다.

프로그램이 멈춘 것처럼 보인다

input()이 사용자 입력을 기다리는 중일 수 있다. 터미널에 값을 입력하고 Enter를 누른다.

반복문이 끝나지 않는다

while True 안에서 break가 실행되는 조건을 확인한다. 강제로 멈춰야 하면 터미널에서 Control + C를 누른다.

부록 E. 초보자 용어 사전

경로(path)

파일이나 폴더가 컴퓨터의 어디에 있는지 나타내는 주소.

프로젝트 폴더

한 프로그램과 관련된 파일을 모아둔 폴더.

터미널

글자로 컴퓨터 명령을 입력하는 앱.

Python 인터프리터

Python 코드를 읽고 실행하는 프로그램.

변수

값을 기억하기 위해 붙인 이름.

리스트

여러 값을 순서대로 담는 자료 구조.

조건문

조건에 따라 다른 코드를 실행하는 구조. if, elif, else.

반복문

코드를 반복하는 구조. while, for.

예외

프로그램 실행 중 예상과 다른 입력이나 상황으로 발생한 오류.

테스트

프로그램이 기대한 대로 동작하는지 여러 입력으로 확인하는 과정.

프롬프트

AI에게 주는 요청이나 지시.

리포지터리(repository)

프로젝트 파일과 변경 기록을 관리하는 저장소.

커밋(commit)

현재 변경 상태를 설명과 함께 기록하는 것.

README

프로젝트의 목적, 기능, 실행 방법을 설명하는 첫 안내 문서.

부록 F. 공식 자료와 확인일

이 책의 설치 및 제품 안내는 2026년 7월 20일에 다음 공식 자료를 확인해 작성했다. 앱과 버전은 계속 바뀔 수 있으므로 실제 화면이 다르면 최신 공식 문서를 우선한다.

1. Python Downloads: <https://www.python.org/downloads/>

2. Python on macOS: <https://docs.python.org/3/using/mac.html>
3. ChatGPT Download: <https://chatgpt.com/download/>
4. Moving to the new ChatGPT desktop app:
<https://help.openai.com/en/articles/20001276-moving-to-the-new-chatgpt-desktop-app>
5. Uploading a project to GitHub: <https://docs.github.com/en/get-started/start-your-journey/uploading-a-project-to-github>
6. Creating your first repository using GitHub Desktop:
<https://docs.github.com/en/desktop/overview/creating-your-first-repository-using-github-desktop>

: Python Software Foundation, “Download Python.” 2026 년 7 월 20 일 확인.

Python Software Foundation, “Using Python on macOS.” 2026 년 7 월 20 일 확인.

OpenAI, “Download ChatGPT.” 2026 년 7 월 20 일 확인.

OpenAI Help Center, “Moving to the new ChatGPT desktop app.” 2026 년 7 월 20 일 확인.

GitHub Docs, “Uploading a project to GitHub.” 2026 년 7 월 20 일 확인.

GitHub Docs, “Creating your first repository using GitHub Desktop.” 2026 년 7 월 20 일 확인.